Neural Inverse Planning in a Gridworld

Why this matters

An important tool for **rational cooperative AI** is **inverse planning**: inferring another agent's hidden goal from its observed behavior.

This matters because cooperative systems often need to reason about what other agents are trying to do before they can coordinate, assist, or avoid conflict.

In this project, we ask:

> **Can we efficiently approximate inverse planning with a neural network?**

Notebook roadmap

This notebook teaches four things:

1. how to define a gridworld inverse-planning task;
2. how to generate trajectories from a Boltzmann-rational agent;
3. how to compute exact Bayesian goal inference from those trajectories;
4. how to train neural networks to approximate that inference process.

The notebook then evaluates those models under three questions: same-layout generalization, random-layout generalization, and rationality ablation.

How trajectories and goal inference are produced

The notebook uses the `memo`-backed exact planner to generate trajectories and Bayesian goal labels.

The key pieces are:

- `sample_trajectory(task, horizon, rng, policy_backend=...)` samples a trajectory from the Boltzmann-rational policy.
- `online_goal_posteriors(task, action_indices, policy_backend=...)` computes the Bayesian posterior after each observed action.

- `exact_goal_posterior(task, action_indices, policy_backend=...)` computes the final Bayesian posterior after the whole trajectory.

Conceptually:

1. `memo` computes exact soft-optimal action values;
2. those values define a Boltzmann-rational policy;
3. `sample_trajectory(...)` generates behavior from that policy;
4. exact Bayesian inference turns that behavior into goal-posterior targets.

The Problem

An agent moves through an **8×8 gridworld** toward one of three possible goals. We observe the states and actions, but the goal is hidden.

The inverse-planning problem is to infer that hidden goal from behaviour. In this setting, a trajectory is evidence: a sequence that repeatedly moves north-west should shift probability toward the north-west goal and away from the south-east goal.

Rational Agents

The data are generated by a Boltzmann-rational agent. For goal g, the agent samples actions according to soft optimal Q-values:

$$\pi(a \mid s, g) \propto \exp(\beta Q^*(s, a, g)).$$

The rationality parameter β controls how deterministic the policy is. Larger β gives more goal-directed actions; smaller β gives noisier trajectories.

Given the policy, an exact Bayesian observer updates the posterior after observing actions $a_{1:T}$:

$$P(g \mid a_{1:T}) \propto P(g) \prod_{t=1}^{T} \pi(a_t \mid s_t, g).$$

This exact posterior is the target used for evaluation.

Learning Setup

Exact inverse planning gives a principled Bayesian observer, but it can be computationally expensive.

So instead of running exact inference directly at test time, we train neural models to approximate the observer's posterior from trajectory data. Each model sees the same sequence of grid frames and previous actions, and predicts a posterior over goals at each timestep.

The notebook is organized around three questions:

- **Q1:** Do the models generalize to held-out trajectories from the same layout?
- **Q2:** Do the models generalize across random goal layouts, and does more training data help?
- **Q3:** How does the agent's rationality change the inference problem?

Models

We compare three architectures.

Action-GRU. Flattens each grid frame, concatenates a previous-action embedding, and processes the sequence with a GRU. It has access to the full state, but no spatial inductive bias.

Conv-GRU. Uses a small convolutional encoder before the GRU. This tests whether a spatial encoder helps when goal locations and obstacles matter.

Goal-conditioned policy. Learns $\hat{\pi}(a \mid s, g)$ for each candidate goal, then reconstructs the posterior by scoring the observed actions under each goal:

$$P(g_i \mid a_{1:T}) \propto \prod_{t=1}^{T} \hat{\pi}(a_t \mid s_t, g_i).$$

This model is closest to the data-generating process: it learns action likelihoods first, then applies the same Bayesian structure used by the exact observer.

Table of Contents

#	Section	Purpose
1	Setup	Imports, task configuration, GPU selection, helper functions
2	Grid & Baseline Trajectory	Show the environment and one exact posterior update
3	Scenario Walkthroughs	Build intuition with fixed trajectories
4	Question 1: Can models generalize to held-out trajectories?	Train and test on the default layout
5	Question 2: Can models generalize across diverse layouts and goals?	Test random-layout generalization and data scaling

#	Section	Purpose
6	Question 3: How does rationality change the task?	Vary β while keeping the Q1 layout fixed

1. Setup

The setup cell imports the project code, selects the visible GPUs, creates the default task, and sets the shared training/evaluation constants.

Select the GPUs and define the default task. The notebook uses `CUDA_VISIBLE_DEVICES` so runs can be restricted to a specific subset of physical GPUs.

In [1]:
```
import os

# Restrict which physical GPUs this notebook can use.
# Examples: '1,3' => GPUs 1 and 3
GPU_IDS = '0,1,6,7'
os.environ['CUDA_VISIBLE_DEVICES'] = GPU_IDS

import numpy as np
import matplotlib.pyplot as plt
import torch
from dataclasses import replace
from pathlib import Path
from IPython.display import HTML, SVG, display

from inverse_planning.data import collect_dataset, encode_grid_frame, save_c
from inverse_planning.inference import (
    exact_goal_posterior,
    online_goal_posteriors,
    online_posteriors_from_goal_conditioned_action_probs,
    posterior_from_goal_conditioned_scores,
    score_goal_conditioned_policy,
)
from inverse_planning.memo_backend import MemoPolicyBackend
from inverse_planning.rnn_models import ModelFactory
from inverse_planning.simulate import sample_trajectory
from inverse_planning.task import GridworldTask, make_default_task
from inverse_planning.training import (
    TrainingConfig,
    build_classifier_loader,
    build_policy_loader,
    evaluate_classifier,
    evaluate_policy_model,
    train_classifier,
    train_policy_model,
)
from inverse_planning.visualize import render_gridworld_svg, render_trajectc

# Shared tutorial settings.
```

```
HORIZON = 12
DEFAULT_TRAIN_EPISODES = 1024
DEFAULT_EVAL_EPISODES = 512
# Training settings.  The earlier GPU-optimized batch=256 setting made Q1 ur
# because it produced too few optimizer steps; batch=128 + more epochs keeps
# while still using reasonably large GPU batches.
DEFAULT_EPOCHS = 24
BUDGET_EPOCHS = 8
DEFAULT_BATCH_SIZE = 128
EVAL_BATCH_SIZE = 512

# Default task.
task = make_default_task(beta=2.0)
memo_backend = MemoPolicyBackend(task)
memo_backend.build()
device = 'cuda:0' if torch.cuda.is_available() else 'cpu'
USE_DATA_PARALLEL = torch.cuda.is_available() and torch.cuda.device_count()
if torch.cuda.is_available():
    torch.backends.cudnn.benchmark = True
    torch.set_float32_matmul_precision('high')
artifacts_dir = Path('artifacts')
artifacts_dir.mkdir(parents=True, exist_ok=True)

print('task shape :', task.shape)
print('goals      :', task.goal_locs)
print('visible GPUs:', GPU_IDS or '(CPU only)')
print('device      :', device)
print('cuda devices:', torch.cuda.device_count())
print('data parallel:', USE_DATA_PARALLEL)
```

```
task shape : (8, 8)
goals      : ((0, 0), (0, 7), (7, 7))
visible GPUs: 0,1,6,7
device      : cuda:0
cuda devices: 4
data parallel: True
```

Define shared plot styles so figures use the same model labels and colors throughout.

In [2]:

```
# Plot style used throughout the notebook.
MODEL_STYLES = {
    'action_gru':              {'label': 'Action-GRU',              'color':
    'conv_gru':                {'label': 'Conv-GRU',                'color':
    'goal_conditioned_policy': {'label': 'Goal-Conditioned Policy', 'color':
}
BETA_STYLES = {
    0.75: {'label': 'beta = 0.75', 'color': '#66a61e', 'marker': 'o'},
    1.5:  {'label': 'beta = 1.5',  'color': '#e6ab02', 'marker': '^'},
    3.0:  {'label': 'beta = 3.0',  'color': '#a6761d', 'marker': 's'},
}
LAYOUT_STYLES = {
    'corners': {'label': 'Corner Goals (default)', 'color': '#1f78b4'},
    'cluster': {'label': 'Central Cluster',        'color': '#33a02c'},
    'right':   {'label': 'Right-Side Spread',      'color': '#e31a1c'},
}
```

```
plt.rcParams.update({
    'axes.spines.top': False,
    'axes.spines.right': False,
    'axes.titlesize': 12,
    'axes.labelsize': 11,
    'legend.frameon': False,
})
```

Helper functions for dataset collection, random-layout sampling, model training, evaluation, and multi-GPU wrapping.

In [3]:
```
def collect_and_save(name, task_i, backend_i, episodes, seed, horizon=HORIZC
    """Collect one dataset from one fixed goal layout and save it under arti
    bundle = collect_dataset(
        task_i,
        n_episodes=episodes,
        horizon=horizon,
        seed=seed,
        policy_backend=backend_i,
    )
    path = artifacts_dir / f'{name}.npz'
    save_dataset(path, bundle)
    unique_goals, counts = np.unique(bundle.goals, return_counts=True)
    print(f'{name:<28} {episodes:>5} episodes | goal counts: {dict(zip(uniqu
    return bundle, np.load(path), path


def concatenate_bundles(bundles):
    """Merge several DatasetBundle objects that share the same shape and num
    return DatasetBundle(
        grids=np.concatenate([b.grids for b in bundles], axis=0),
        positions=np.concatenate([b.positions for b in bundles], axis=0),
        actions=np.concatenate([b.actions for b in bundles], axis=0),
        goals=np.concatenate([b.goals for b in bundles], axis=0),
        final_posteriors=np.concatenate([b.final_posteriors for b in bundles
        online_posteriors=np.concatenate([b.online_posteriors for b in bundl
        goal_condition_inputs=np.concatenate([b.goal_condition_inputs for b
    )


def sample_goal_layouts(base_task, n_layouts, seed, n_goals=None):
    """Sample random goal layouts from non-wall cells, excluding the start c
    rng = np.random.default_rng(seed)
    n_goals = base_task.n_goals if n_goals is None else n_goals
    open_cells = [tuple(cell) for cell in np.argwhere(~base_task.grid) if tu
    layouts = []
    seen = set()
    while len(layouts) < n_layouts:
        indices = rng.choice(len(open_cells), size=n_goals, replace=False)
        layout = tuple(sorted(open_cells[i] for i in indices))
        if layout not in seen:
            layouts.append(layout)
            seen.add(layout)
    return layouts
```

```
def collect_random_layout_dataset(name, base_task, episodes, n_layouts, seec
    """Collect a dataset from randomly sampled goal layouts.

    Each layout is random; within each layout, the true goal for each episoc
    by the trajectory simulator. Train and test calls should use different s
    held-out layouts are new.
    """
    beta = base_task.beta if beta is None else beta
    n_layouts = min(n_layouts, episodes)
    layouts = sample_goal_layouts(base_task, n_layouts=n_layouts, seed=seed)
    episode_counts = np.full(n_layouts, episodes // n_layouts, dtype=int)
    episode_counts[: episodes % n_layouts] += 1

    parts = []
    for layout_idx, (goal_locs, n_eps) in enumerate(zip(layouts, episode_cou
        layout_task = GridworldTask(grid=base_task.grid, init_loc=base_task.
        layout_task.validate()
        backend = MemoPolicyBackend(layout_task)
        backend.build()
        part = collect_dataset(
            layout_task,
            n_episodes=int(n_eps),
            horizon=horizon,
            seed=seed + 10_000 + layout_idx,
            policy_backend=backend,
        )
        parts.append(part)

    bundle = concatenate_bundles(parts)
    path = artifacts_dir / f'{name}.npz'
    save_dataset(path, bundle)
    print(f'{name:<28} {episodes:>5} episodes | {n_layouts:>3} random layout
    return bundle, np.load(path), path, layouts



def maybe_parallel(model):
    """Use all visible GPUs for the model when more than one CUDA device is
    if USE_DATA_PARALLEL:
        return torch.nn.DataParallel(model)
    return model


def save_model_state(model, path):
    """Save the underlying module when DataParallel is active."""
    stateful = model.module if isinstance(model, torch.nn.DataParallel) else
    torch.save(stateful.state_dict(), path)


def materialize_lazy_model(model, train_npz, *, policy=False, start_token=Nc
    """Run one tiny forward pass before DataParallel so LazyLinear parameter
    model = model.to(device)
    frames = torch.tensor(train_npz['grids'][:1], dtype=torch.float32, devic
    actions = np.asarray(train_npz['actions'][:1], dtype=np.int64)
    start_token = int(actions.max()) + 1 if start_token is None else int(sta
```

```
    prev_actions = np.full_like(actions, start_token)
    prev_actions[:, 1:] = actions[:, :-1]
    prev_actions_t = torch.tensor(prev_actions, dtype=torch.long, device=dev
    with torch.no_grad():
        if policy:
            goal_ids = torch.zeros(1, dtype=torch.long, device=device)
            model(frames, goal_ids, prev_actions_t)
        else:
            model(frames, prev_actions_t)
    return model


def train_model_suite(train_npz, eval_npz=None, epochs=DEFAULT_EPOCHS, prefi
    """Train the two discriminative GRUs and the goal-conditioned policy moc
    n_actions = max(task.n_actions, int(train_npz['actions'].max()) + 1)
    if eval_npz is not None:
        n_actions = max(n_actions, int(eval_npz['actions'].max()) + 1)
    factory = ModelFactory(
        in_channels=train_npz['grids'].shape[2],
        n_actions=n_actions,
        n_goals=train_npz['final_posteriors'].shape[-1],
    )
    config = TrainingConfig(batch_size=DEFAULT_BATCH_SIZE, epochs=epochs, le
    runs = {}

    for variant in ['action_gru', 'conv_gru']:
        torch.manual_seed(0)
        model = materialize_lazy_model(factory.build_classifier(variant), tr
        model = maybe_parallel(model)
        losses = train_classifier(model, build_classifier_loader(train_npz,
        metrics = evaluate_classifier(model, eval_npz, device=device, start_
        runs[variant] = {'model': model, 'losses': losses, 'metrics': metric
        save_model_state(model, artifacts_dir / f'{prefix}_{variant}.pt')

    torch.manual_seed(0)
    policy_model = materialize_lazy_model(factory.build_policy_model(), trai
    policy_model = maybe_parallel(policy_model)
    policy_losses = train_policy_model(policy_model, build_policy_loader(tra
    policy_metrics = evaluate_policy_model(policy_model, eval_npz, device=de
    runs['goal_conditioned_policy'] = {'model': policy_model, 'losses': poli
    save_model_state(policy_model, artifacts_dir / f'{prefix}_goal_conditior

    return runs


def print_run_table(runs):
    print(f"{'model':<28} {'final loss':>10} {'final-goal acc':>14} {'poster
    print('-' * 70)
    for name, run in runs.items():
        metrics = run.get('metrics', {})
        acc = metrics.get('final_goal_accuracy', float('nan'))
        kl = metrics.get('posterior_kl', float('nan'))
        print(f"{name:<28} {run['losses'][-1]:>10.4f} {acc:>14.4f} {kl:>12.4
```

2. Grid & Baseline Trajectory

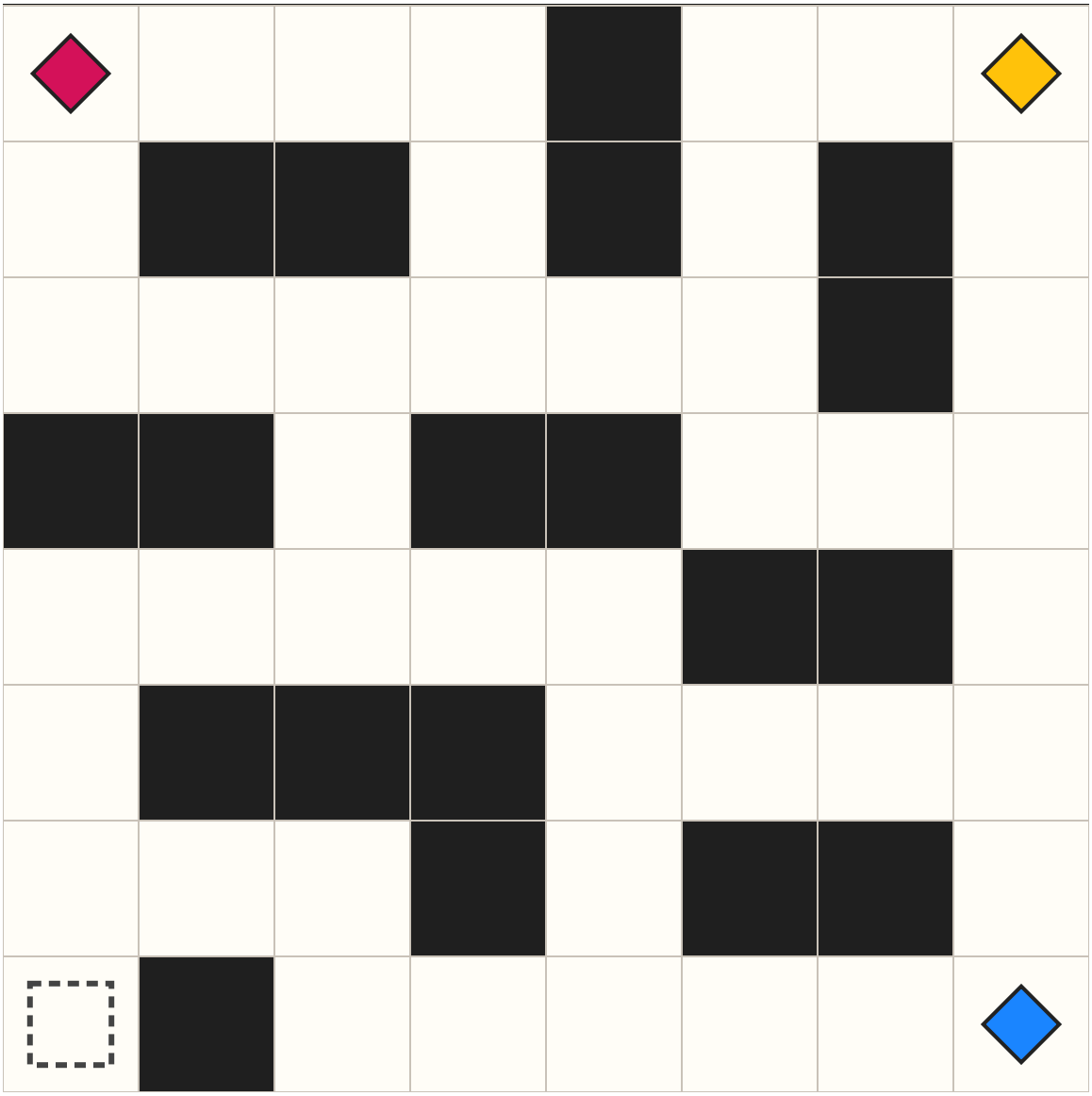
This section displays the default grid and one trajectory from the exact memoized policy. The posterior printed below is the Bayesian reference that the learned models try to approximate.

Render the default grid, sample one trajectory, and print the exact posterior after each observed action.

In [4]:
```
print('initial location:', task.init_loc)
print(task.grid.astype(int))
display(SVG(render_gridworld_svg(task, cell_size=72, title='Default 8x8 gric

baseline = sample_trajectory(task, horizon=12, rng=np.random.default_rng(0),
baseline_posterior = exact_goal_posterior(task, baseline.action_indices, pol
baseline_online   = online_goal_posteriors(task, baseline.action_indices, pc

print('sampled goal index :', baseline.goal_index)
print('sampled actions    :', baseline.action_indices)
print('final exact posterior:', np.round(baseline_posterior, 3))
display(SVG(render_gridworld_svg(task, trajectory=baseline, cell_size=72, ti
print('online exact posterior (rows = timesteps, cols = goals):')
print(np.round(baseline_online, 3))
```

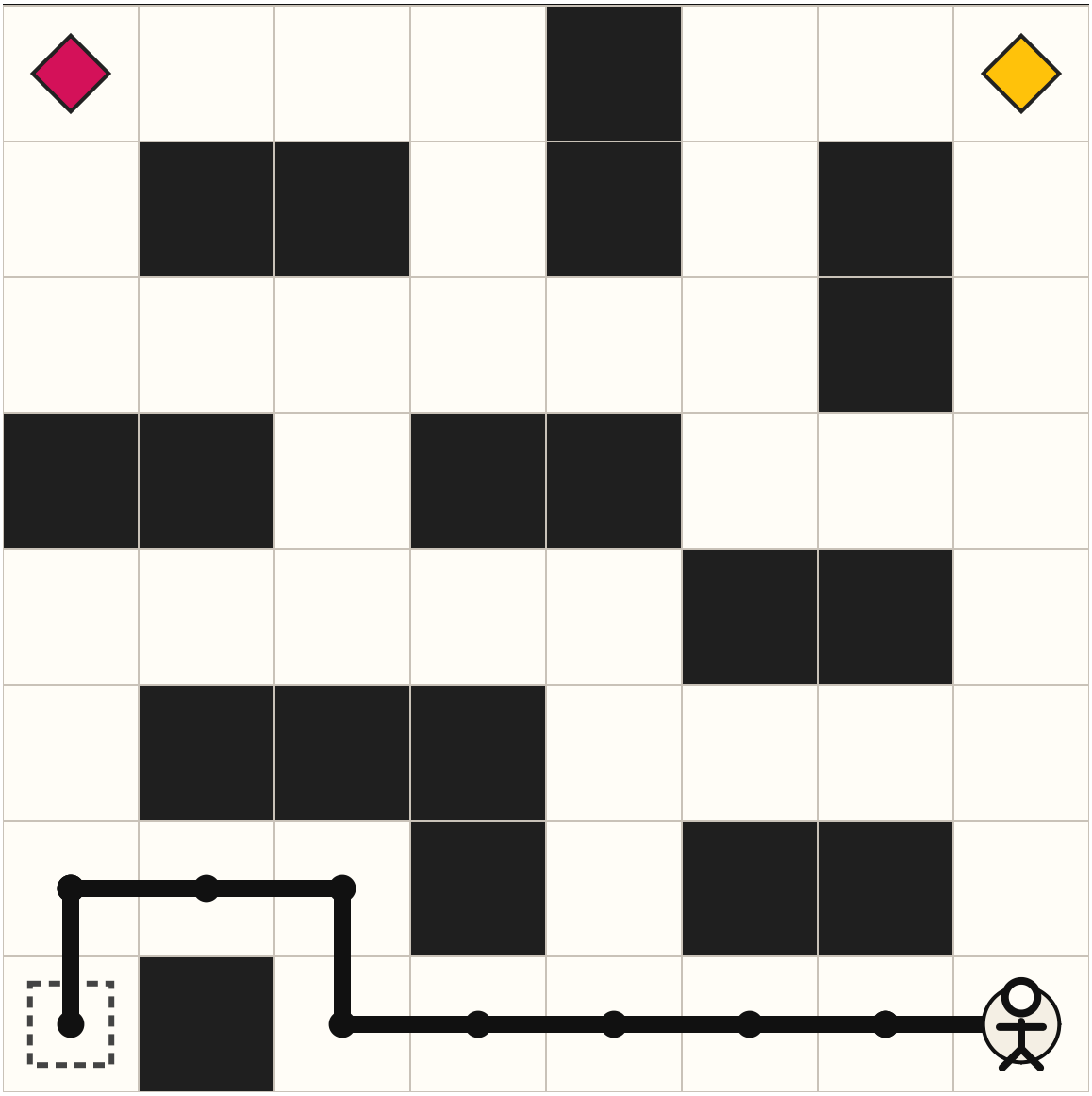
```
initial location: (7, 0)
[[0 0 0 0 1 0 0 0]
 [0 1 1 0 1 0 1 0]
 [0 0 0 0 0 0 1 0]
 [1 1 0 1 1 0 0 0]
 [0 0 0 0 0 1 1 0]
 [0 1 1 1 0 0 0 0]
 [0 0 0 1 0 1 1 0]
 [0 1 0 0 0 0 0 0]]
```

Default 8x8 gridworld



```
sampled goal index : 2
sampled actions    : [1, 0, 0, 4, 4, 2, 4, 4, 4, 4, 0, 4]
final exact posterior: [0.    0.005 0.995]
```

Memo-planned baseline trajectory



```
online exact posterior (rows = timesteps, cols = goals):
[[0.333 0.333 0.333]
 [0.333 0.333 0.333]
 [0.333 0.333 0.333]
 [0.018 0.018 0.965]
 [0.    0.01  0.99 ]
 [0.    0.01  0.99 ]
 [0.    0.01  0.99 ]
 [0.    0.01  0.99 ]
 [0.    0.005 0.995]
 [0.    0.005 0.995]
 [0.    0.005 0.995]
 [0.    0.005 0.995]]
```

3. Scenario Walkthroughs

Before training models, we inspect three fixed trajectories. These examples show when the exact observer can disambiguate goals and when two goals remain

plausible for several steps.

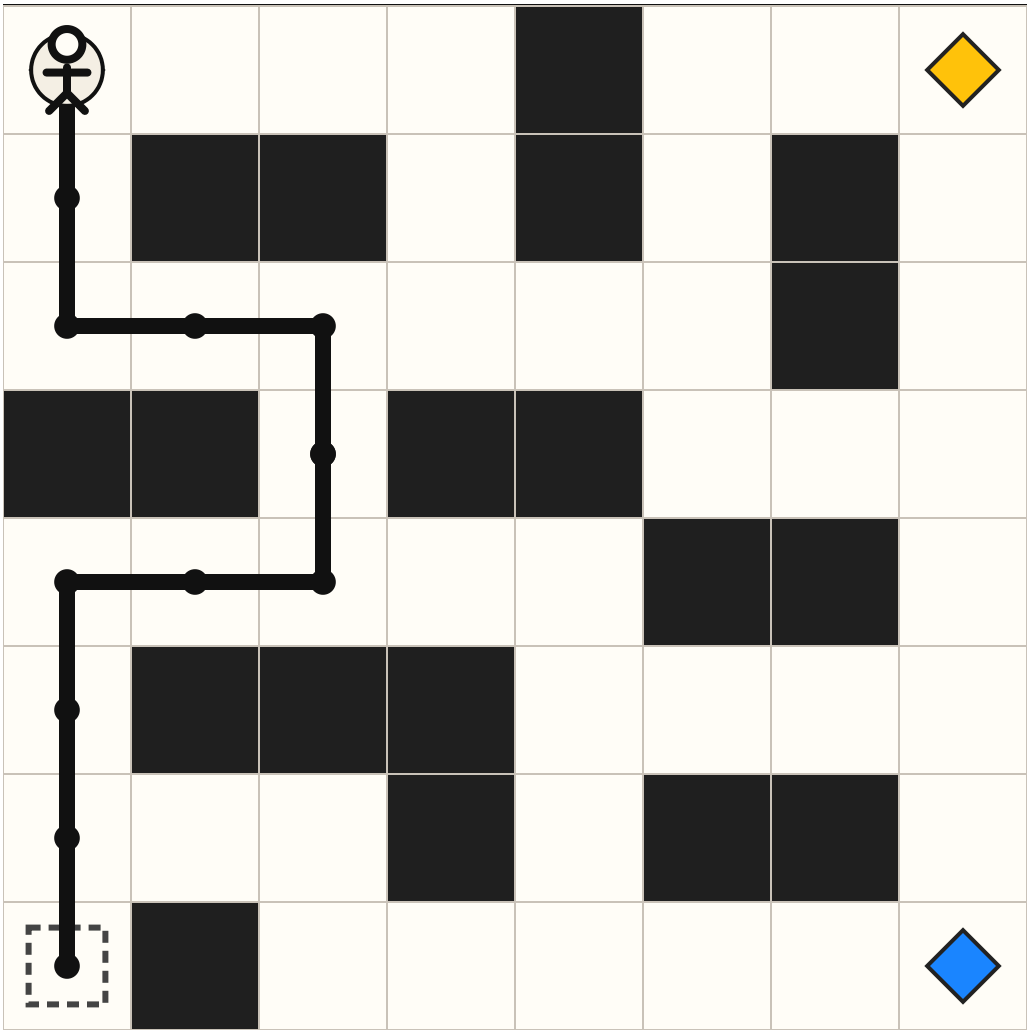
Scenario	Start	True goal	What to notice
Northwest chase	(7, 0)	0 — (0, 0)	Early northward moves still leave two top goals plausible; later westward moves resolve the goal.
Cross-map east goal	(4, 0)	1 — (0, 7)	The posterior flips once the trajectory commits to the east side.
Bottom-right commitment	(6, 2)	2 — (7, 7)	Evidence accumulates gradually before the posterior becomes decisive.

Run the three fixed scenarios and print the exact online posterior for each one.

In [5]:
```
scenario_specs = [
    {'name': 'Northwest chase',        'init_loc': (7, 0), 'goal_index': 0,
    {'name': 'Cross-map east goal',    'init_loc': (4, 0), 'goal_index': 1,
    {'name': 'Bottom-right commitment','init_loc': (6, 2), 'goal_index': 2,
]

scenario_runs = []
for spec in scenario_specs:
    scenario_task = replace(task, init_loc=spec['init_loc'])
    scenario_task.validate()
    trajectory = sample_trajectory(
        scenario_task,
        horizon=spec['horizon'],
        rng=np.random.default_rng(spec['seed']),
        goal_index=spec['goal_index'],
        policy_backend=memo_backend,
    )
    posterior = exact_goal_posterior(scenario_task, trajectory.action_indice
    online    = online_goal_posteriors(scenario_task, trajectory.action_indi
    scenario_runs.append((spec, scenario_task, trajectory, posterior, online

    print(f"=== {spec['name']} ===")
    print(f"  start={spec['init_loc']}  goal={scenario_task.goal_locs[spec['
    print('  actions        :', trajectory.action_indices)
    print('  final posterior:', np.round(posterior, 3))
    display(SVG(render_gridworld_svg(scenario_task, trajectory=trajectory, c
    print('  online posterior (rows = timesteps):')
    print(np.round(online, 3))
    print()
```

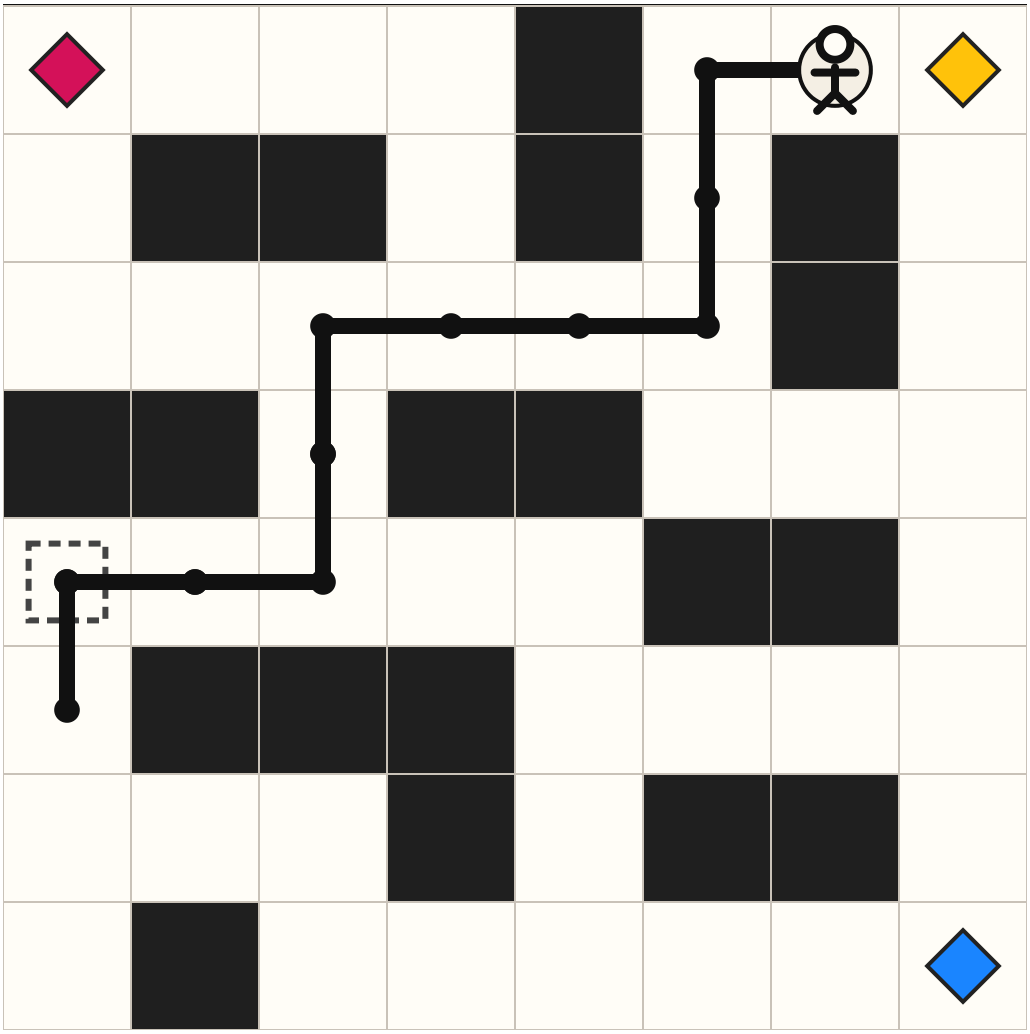
```
=== Northwest chase ===
  start=(7, 0)  goal=(0, 0)
  actions        : [1, 1, 1, 4, 4, 1, 0, 1, 3, 3, 1, 1]
  final posterior: [1. 0. 0.]
```

Northwest chase



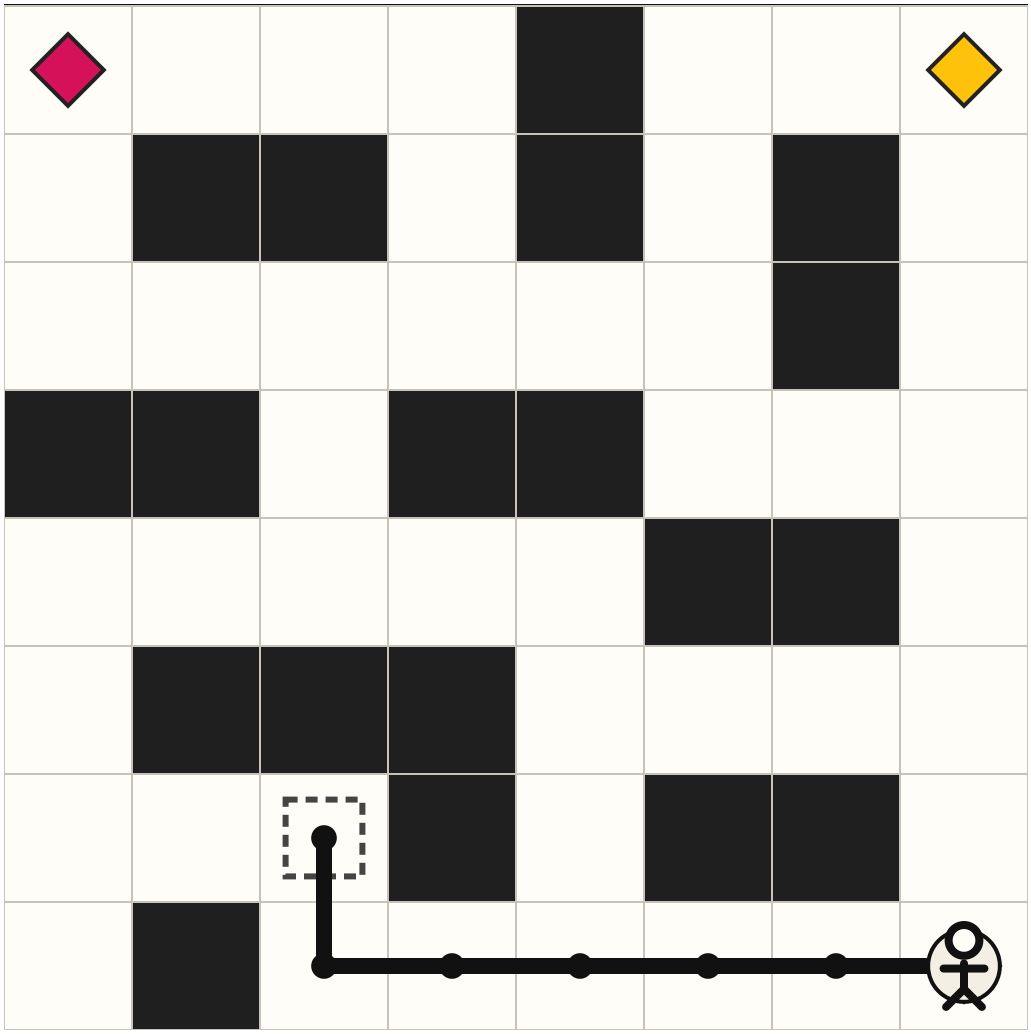
```
  online posterior (rows = timesteps):
[[0.333 0.333 0.333]
 [0.495 0.495 0.009]
 [0.5   0.5   0.   ]
 [0.5   0.5   0.   ]
 [0.5   0.5   0.   ]
 [0.5   0.5   0.   ]
 [0.5   0.5   0.   ]
 [0.5   0.5   0.   ]
 [0.982 0.018 0.   ]
 [1.    0.    0.   ]
 [1.    0.    0.   ]
 [1.    0.    0.   ]]

=== Cross-map east goal ===
  start=(4, 0)  goal=(0, 7)
  actions        : [2, 1, 4, 0, 4, 1, 0, 1, 4, 4, 4, 1, 1, 4]
  final posterior: [0. 1. 0.]
```

Cross-map east goal



```
  online posterior (rows = timesteps):
[[0.032 0.032 0.937]
 [0.394 0.394 0.213]
 [0.436 0.436 0.127]
 [0.436 0.436 0.127]
 [0.436 0.436 0.127]
 [0.499 0.499 0.003]
 [0.499 0.499 0.003]
 [0.5   0.5   0.   ]
 [0.018 0.982 0.   ]
 [0.    1.    0.   ]
 [0.    1.    0.   ]
 [0.    1.    0.   ]
 [0.    1.    0.   ]
 [0.    1.    0.   ]]

=== Bottom-right commitment ===
  start=(6, 2)  goal=(7, 7)
  actions        : [2, 4, 4, 4, 4, 4, 0, 0, 0, 0]
  final posterior: [0. 0. 1.]
```

Bottom-right commitment



```
  online posterior (rows = timesteps):
[[0.009 0.495 0.495]
 [0.005 0.498 0.498]
 [0.005 0.498 0.498]
 [0.    0.352 0.648]
 [0.    0.352 0.648]
 [0.    0.352 0.648]
 [0.    0.075 0.925]
 [0.    0.012 0.988]
 [0.    0.002 0.998]
 [0.    0.    1.   ]]
```

4. Question 1: Can models generalize to held-out trajectories?

Q1 is the in-distribution test. The models train on trajectories from the default corner-goal layout and evaluate on new trajectories from the same layout.

This question uses 1024 training episodes and 512 held-out evaluation episodes.

In [6]:
```
# One default training set and one held-out evaluation set for Q1.
default_train_bundle, default_train_npz, default_train_path = collect_and_sa
    'q1_default_train_1024', task, memo_backend, DEFAULT_TRAIN_EPISODES, see
)
default_eval_bundle, default_eval_npz, default_eval_path = collect_and_save(
    'q1_default_eval_512', task, memo_backend, DEFAULT_EVAL_EPISODES, seed=9
)
```

```
q1_default_train_1024         1024 episodes | goal counts: {0: 348, 1: 351,
2: 325}
q1_default_eval_512            512 episodes | goal counts: {0: 172, 1: 156,
2: 184}
```

Train all three model families on the Q1 dataset. We report final-goal accuracy and posterior KL; lower posterior KL means the predicted posterior is closer to the exact Bayesian posterior.

In [7]:
```
default_runs = train_model_suite(
    default_train_npz,
    eval_npz=default_eval_npz,
    epochs=DEFAULT_EPOCHS,
    prefix='q1_default',
)
print_run_table(default_runs)
```

```
Warning: Attempting to run cuBLAS, but there was no current CUDA context! At
tempting to set the primary context... (Triggered internally at /pytorch/ate
n/src/ATen/cuda/CublasHandlePool.cpp:330.)
model                        final loss final-goal acc posterior KL
----------------------------------------------------------------------
action_gru                       1.3053         0.9668       0.0951
conv_gru                         1.3901         0.9707       0.1007
goal_conditioned_policy          0.6698         0.9844       0.0536
```

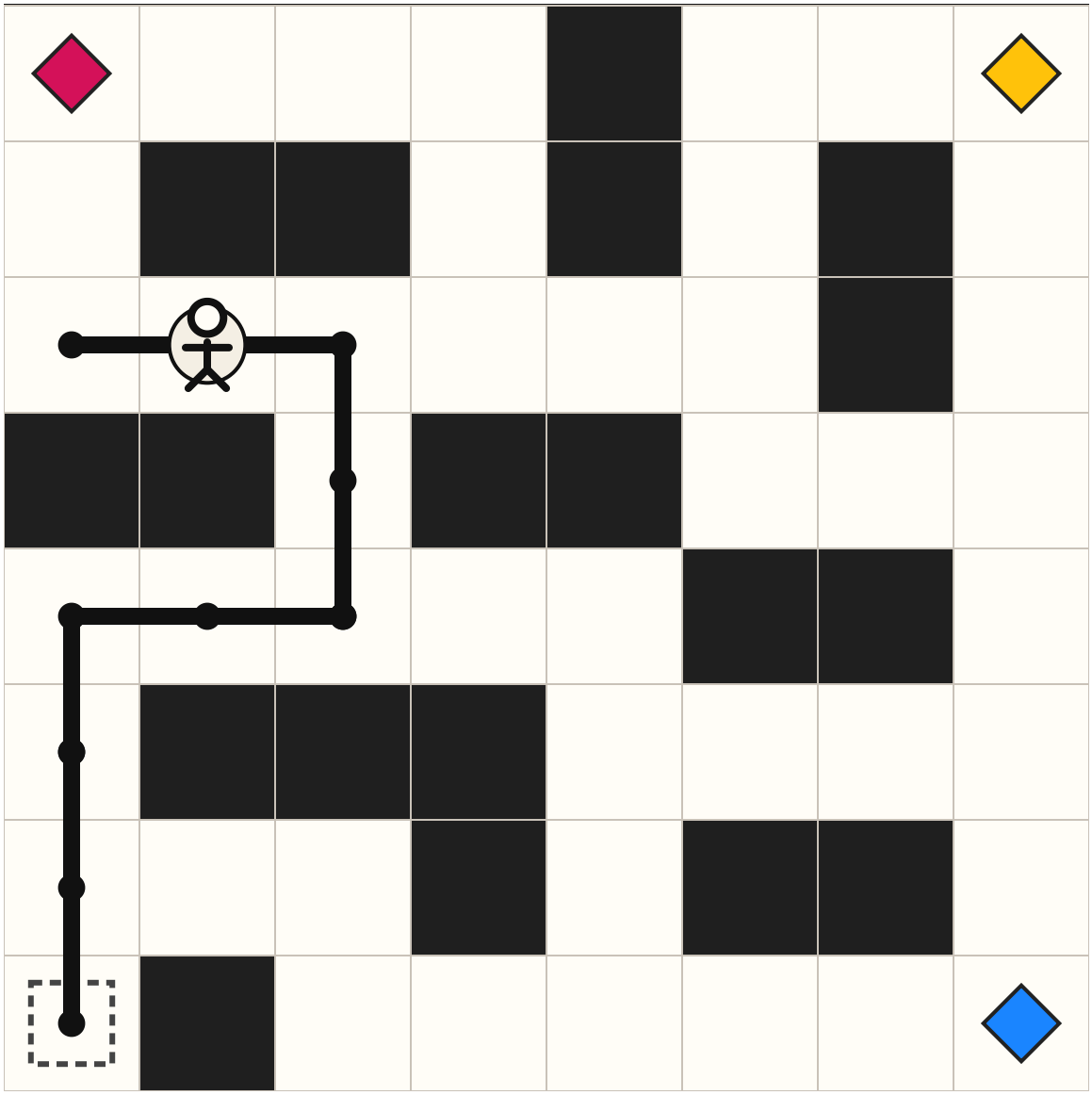
Inspect one held-out Q1 trajectory. The printed final beliefs check whether each model assigns high probability to the same goal as the exact observer.

In [8]:
```
heldout = sample_trajectory(task, horizon=HORIZON, rng=np.random.default_rng
heldout_exact = online_goal_posteriors(task, heldout.action_indices, policy_

frames = np.stack(
    [encode_grid_frame(task, position=heldout.positions[step], step=step, ho
     for step in range(len(heldout.action_indices))],
    axis=0,
)[None]
prev_actions = np.full((1, len(heldout.action_indices)), task.n_actions, dty
prev_actions[0, 1:] = np.asarray(heldout.action_indices[:-1], dtype=np.int64
action_indices = np.asarray(heldout.action_indices, dtype=np.int64)
frames_t = torch.tensor(frames, dtype=torch.float32, device=device)
prev_actions_t = torch.tensor(prev_actions, dtype=torch.long, device=device)

for run in default_runs.values():
    run['model'].eval().to(device)
```

```
with torch.no_grad():
    action_gru_out = default_runs['action_gru']['model'](frames_t, prev_acti
    conv_gru_out = default_runs['conv_gru']['model'](frames_t, prev_actions_

action_gru_online = torch.softmax(action_gru_out['posterior_logits'], dim=-1
conv_gru_online = torch.softmax(conv_gru_out['posterior_logits'], dim=-1)[0]
action_gru_final = torch.softmax(action_gru_out['goal_logits'], dim=-1)[0].c
conv_gru_final = torch.softmax(conv_gru_out['goal_logits'], dim=-1)[0].detac

policy_action_probs = []
policy_scores_list = []
with torch.no_grad():
    for goal_index in range(task.n_goals):
        goal_ids = torch.tensor([goal_index], dtype=torch.long, device=devic
        logits = default_runs['goal_conditioned_policy']['model'](frames_t,
        probs = torch.softmax(logits, dim=-1)[0].detach().cpu().numpy()
        policy_action_probs.append(probs)
        policy_scores_list.append(score_goal_conditioned_policy(probs, actic
policy_action_probs = np.asarray(policy_action_probs)
policy_posterior = posterior_from_goal_conditioned_scores(np.asarray(policy_
policy_online = online_posteriors_from_goal_conditioned_action_probs(policy_

print('held-out goal index         :', heldout.goal_index)
print('held-out actions            :', heldout.action_indices)
print('exact final posterior       :', np.round(heldout_exact[-1], 3))
print('action-gru final belief     :', np.round(action_gru_final, 3))
print('conv-gru final belief       :', np.round(conv_gru_final, 3))
print('goal-cond. policy posterior :', np.round(policy_posterior, 3))
display(SVG(render_gridworld_svg(task, trajectory=heldout, cell_size=72, tit
```

```
held-out goal index         : 0
held-out actions            : [1, 1, 0, 1, 4, 4, 0, 1, 1, 3, 3, 4]
exact final posterior       : [0.982 0.018 0.   ]
action-gru final belief     : [0.994 0.006 0.   ]
conv-gru final belief       : [0.996 0.004 0.   ]
goal-cond. policy posterior : [0.979 0.021 0.   ]
```

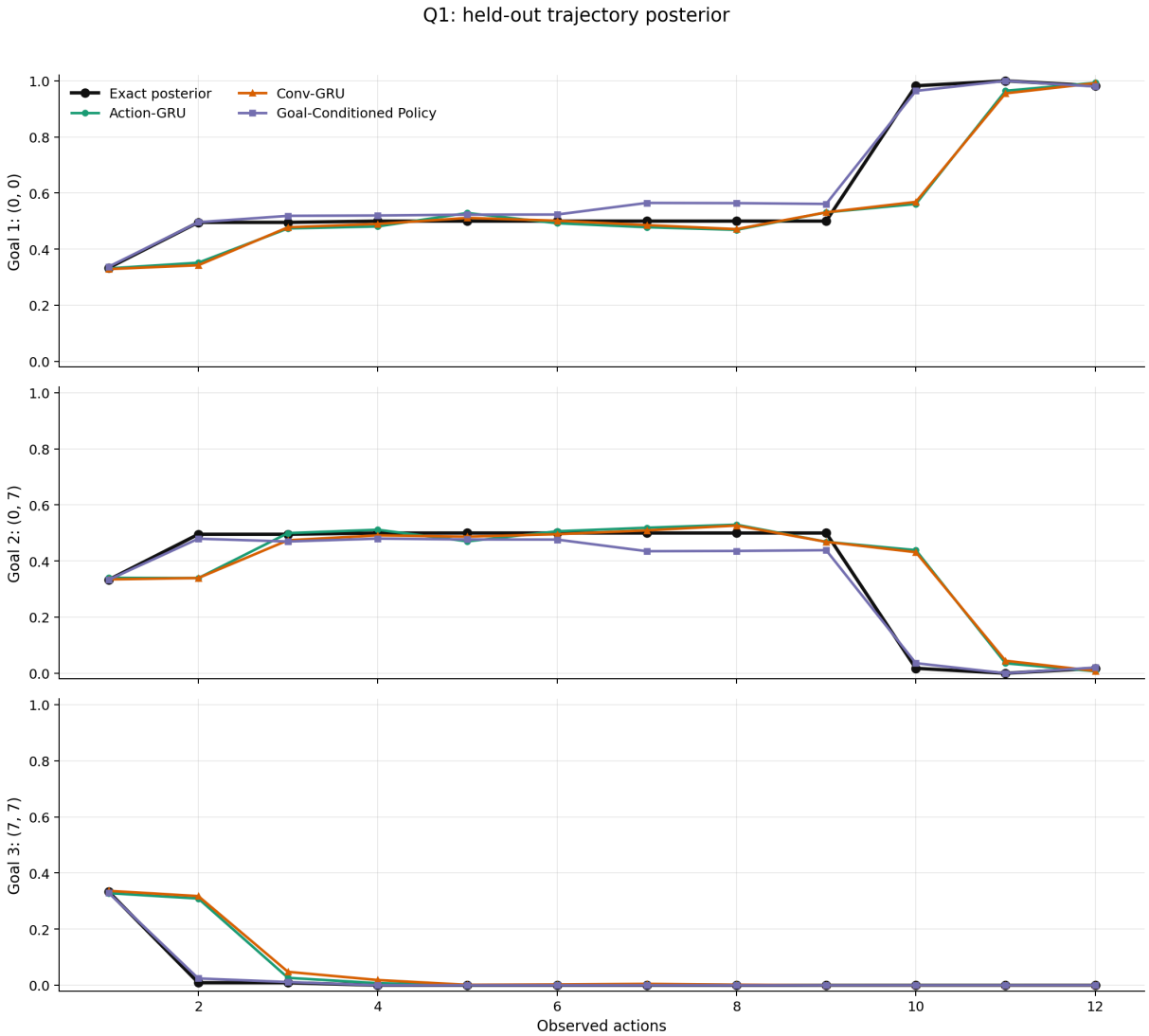
Held-out memo-generated trajectory



Plot the full online posterior for the same held-out trajectory, one panel per candidate goal.

In [9]:
```
timesteps = np.arange(1, len(heldout.action_indices) + 1)
goal_labels = [f'Goal {idx + 1}: {goal}' for idx, goal in enumerate(task.goa
fig, axes = plt.subplots(task.n_goals, 1, figsize=(12, 3.5 * task.n_goals),
if task.n_goals == 1:
    axes = [axes]

for goal_index, ax in enumerate(axes):
    ax.plot(timesteps, heldout_exact[:, goal_index], marker='o', linewidth=2
    for variant, series in [
        ('action_gru', action_gru_online[:, goal_index]),
        ('conv_gru', conv_gru_online[:, goal_index]),
        ('goal_conditioned_policy', policy_online[:, goal_index]),
    ]:
        style = MODEL_STYLES[variant]
        ax.plot(timesteps, series, marker=style['marker'], markersize=4, lir
```

```
    ax.set_ylim(-0.02, 1.02)
    ax.set_ylabel(goal_labels[goal_index])
    ax.grid(alpha=0.25)
    if goal_index == 0:
        ax.legend(loc='upper left', ncol=2)

axes[-1].set_xlabel('Observed actions')
fig.suptitle('Q1: held-out trajectory posterior', y=1.02, fontsize=14)
fig.tight_layout()
plt.show()
```



Q1 conclusion. The same-layout generalization test succeeds. On 512 held-out trajectories, all three models reach high final-goal accuracy, and the goal-conditioned policy model has the lowest posterior KL. The qualitative trajectory agrees with the exact observer: all models put almost all probability on the correct goal by the final step.

The takeaway is that this task is learnable when the goal layout is fixed. In this setting, the remaining challenge is matching the posterior shape over time, not identifying the final goal.

5. Question 2: Can models generalize across diverse layouts and goals?

Q2 tests a harder distribution shift. Training uses 32 random candidate-goal layouts; evaluation uses 32 different random layouts sampled with a separate seed.

We first use the Q1-sized budget, then increase the number of training episodes while keeping the number of distinct training layouts fixed. This asks whether more trajectories from random layouts are enough to improve held-out layout generalization.

Collect the Q2 datasets: one held-out random-layout evaluation set, plus random-layout training sets with increasing episode budgets.

In [10]:
```
q2_budget_specs = [
    {'name': 'q2_random_budget_1024',  'episodes':  1024, 'seed': 3},   # sa
    {'name': 'q2_random_budget_4096',  'episodes':  4096, 'seed': 5},
    {'name': 'q2_random_budget_8192',  'episodes':  8192, 'seed': 6},
    {'name': 'q2_random_budget_16384', 'episodes': 16384, 'seed': 7},
]
Q2_TRAIN_LAYOUTS = 32
Q2_EVAL_LAYOUTS = 32

_, q2_eval_npz, _, q2_eval_layouts = collect_random_layout_dataset(
    'q2_random_eval_512', task, episodes=512, n_layouts=Q2_EVAL_LAYOUTS, see
)

q2_budget_registry = {}
for spec in q2_budget_specs:
    _, budget_npz, budget_path, budget_layouts = collect_random_layout_datas
        spec['name'], task, episodes=spec['episodes'], n_layouts=Q2_TRAIN_LA
    )
    q2_budget_registry[spec['name']] = {
        'spec': spec,
        'npz': budget_npz,
        'path': budget_path,
        'layouts': budget_layouts,
    }
```

```
q2_random_eval_512             512 episodes |  32 random layouts
q2_random_budget_1024         1024 episodes |  32 random layouts
q2_random_budget_4096         4096 episodes |  32 random layouts
q2_random_budget_8192         8192 episodes |  32 random layouts
q2_random_budget_16384       16384 episodes |  32 random layouts
```

Train the three model families for each Q2 budget and store the posterior-KL curves for comparison.

In [11]:
```
q2_scaling_results = []
q2_example_runs = None
```

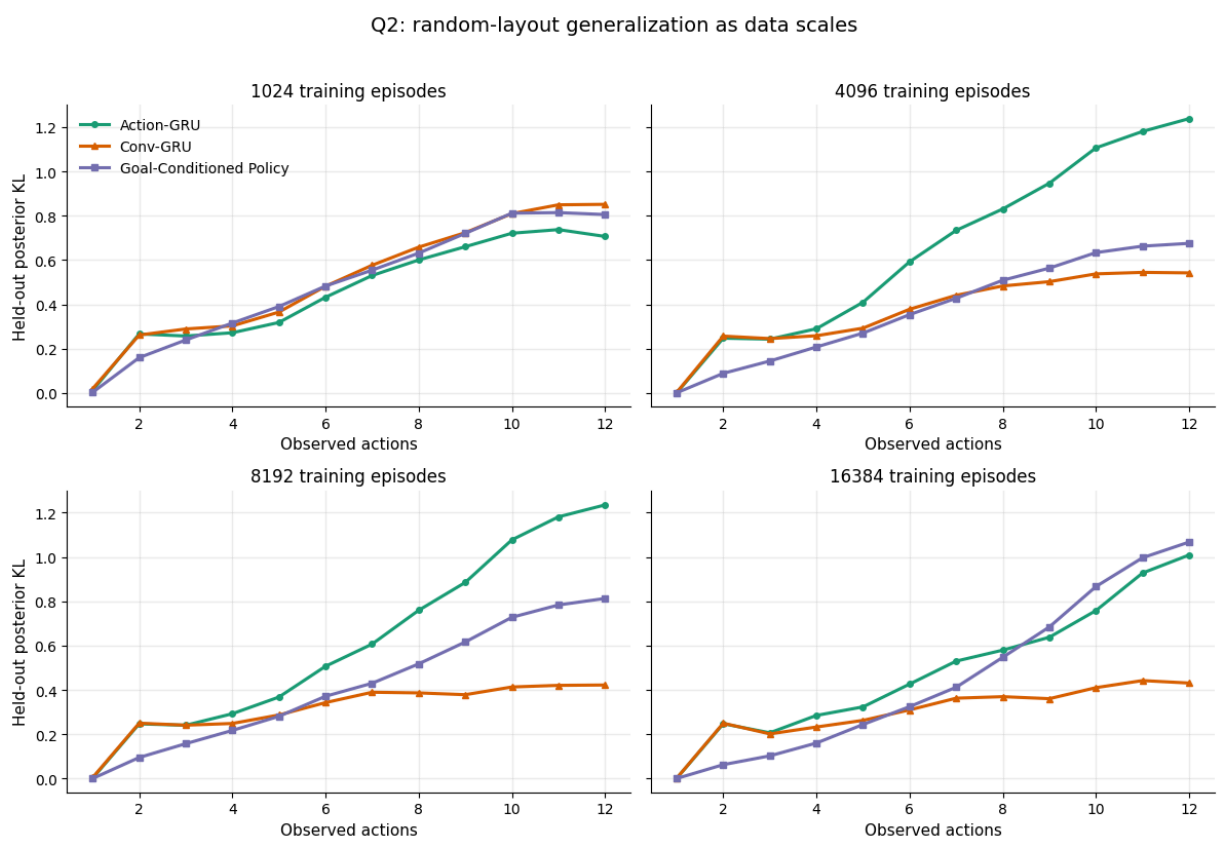
```
q2_example_budget_name = q2_budget_specs[-1]['name']  # use the largest-data

for budget_name, entry in q2_budget_registry.items():
    runs = train_model_suite(
        entry['npz'],
        eval_npz=q2_eval_npz,
        epochs=BUDGET_EPOCHS,
        prefix=budget_name,
    )
    if budget_name == q2_example_budget_name:
        q2_example_runs = runs

    for variant, run in runs.items():
        metrics = run['metrics']
        q2_scaling_results.append({
            'budget_name': budget_name,
            'episodes': int(entry['spec']['episodes']),
            'variant': variant,
            'final_loss': float(run['losses'][-1]),
            'posterior_kl': metrics['posterior_kl'],
            'final_step_kl': metrics['posterior_kl_by_step'][-1],
            'posterior_kl_by_step': metrics['posterior_kl_by_step'],
        })

print(f"{'budget':>22} | {'model':<24} | mean KL | final-step KL")
print('-' * 74)
for row in q2_scaling_results:
    print(f"{row['budget_name']:>22} | {row['variant']:<24} | {row['posteric
```

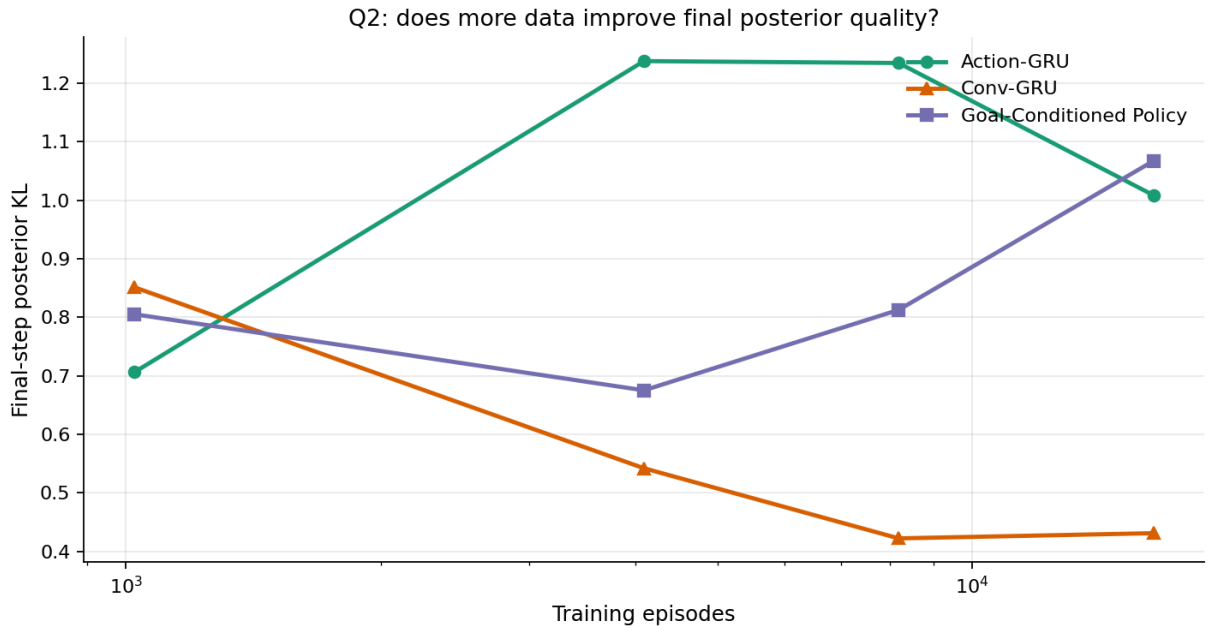
```
                budget | model                    | mean KL | final-step KL
--------------------------------------------------------------------------
 q2_random_budget_1024 | action_gru               | 0.4593  | 0.7064
 q2_random_budget_1024 | conv_gru                 | 0.5155  | 0.8513
 q2_random_budget_1024 | goal_conditioned_policy  | 0.4938  | 0.8053
 q2_random_budget_4096 | action_gru               | 0.6515  | 1.2380
 q2_random_budget_4096 | conv_gru                 | 0.3736  | 0.5420
 q2_random_budget_4096 | goal_conditioned_policy  | 0.3780  | 0.6752
 q2_random_budget_8192 | action_gru               | 0.6166  | 1.2348
 q2_random_budget_8192 | conv_gru                 | 0.3153  | 0.4222
 q2_random_budget_8192 | goal_conditioned_policy  | 0.4175  | 0.8127
q2_random_budget_16384 | action_gru               | 0.4943  | 1.0088
q2_random_budget_16384 | conv_gru                 | 0.3028  | 0.4310
q2_random_budget_16384 | goal_conditioned_policy  | 0.4557  | 1.0673
```

Plot Q2 from two angles: posterior KL over time, and final-step KL as a function of the training budget.

In [12]:
```
# Posterior KL over time for each training-data budget.
fig, axes = plt.subplots(2, 2, figsize=(12, 8), sharey=True)
axes = axes.flatten()
for ax, spec in zip(axes, q2_budget_specs):
    budget_name = spec['name']
    rows = [row for row in q2_scaling_results if row['budget_name'] == budge
    for row in rows:
        style = MODEL_STYLES[row['variant']]
        timesteps = np.arange(1, len(row['posterior_kl_by_step']) + 1)
```

```
        ax.plot(timesteps, row['posterior_kl_by_step'], marker=style['marker
    ax.set_title(f"{spec['episodes']} training episodes")
    ax.set_xlabel('Observed actions')
    ax.grid(alpha=0.25)

axes[0].set_ylabel('Held-out posterior KL')
axes[2].set_ylabel('Held-out posterior KL')
axes[0].legend(loc='upper left', ncol=1)
fig.suptitle('Q2: random-layout generalization as data scales', y=1.02, font
fig.tight_layout()
plt.show()

# Final-step posterior quality as a direct function of training episodes.
fig, ax = plt.subplots(figsize=(9, 4.8), dpi=160)
for variant in ['action_gru', 'conv_gru', 'goal_conditioned_policy']:
    rows = sorted([r for r in q2_scaling_results if r['variant'] == variant]
    episodes = [r['episodes'] for r in rows]
    final_kls = [r['final_step_kl'] for r in rows]
    style = MODEL_STYLES[variant]
    ax.plot(episodes, final_kls, marker=style['marker'], linewidth=2.2, colo
ax.set_xscale('log')
ax.set_xlabel('Training episodes')
ax.set_ylabel('Final-step posterior KL')
ax.set_title('Q2: does more data improve final posterior quality?')
ax.grid(alpha=0.25)
ax.legend(loc='upper right')
fig.tight_layout()
plt.show()
```





Q2 conclusion. Random-layout generalization is substantially harder than Q1. Increasing the data budget helps the Conv-GRU most, but the improvement is not uniform across models. The Action-GRU struggles despite more episodes, and the goal-conditioned policy model is less stable here than it was in the fixed-layout setting.

The takeaway is that more trajectories alone do not fully solve the layout shift. The model must learn how goal geometry changes the meaning of the same action sequence; a spatial encoder helps, but the setting likely needs either more layout diversity or a stronger architecture.

For a qualitative check, use the largest-budget Q2 models on one held-out random-layout trajectory. This layout is from the evaluation set, not the training layouts.

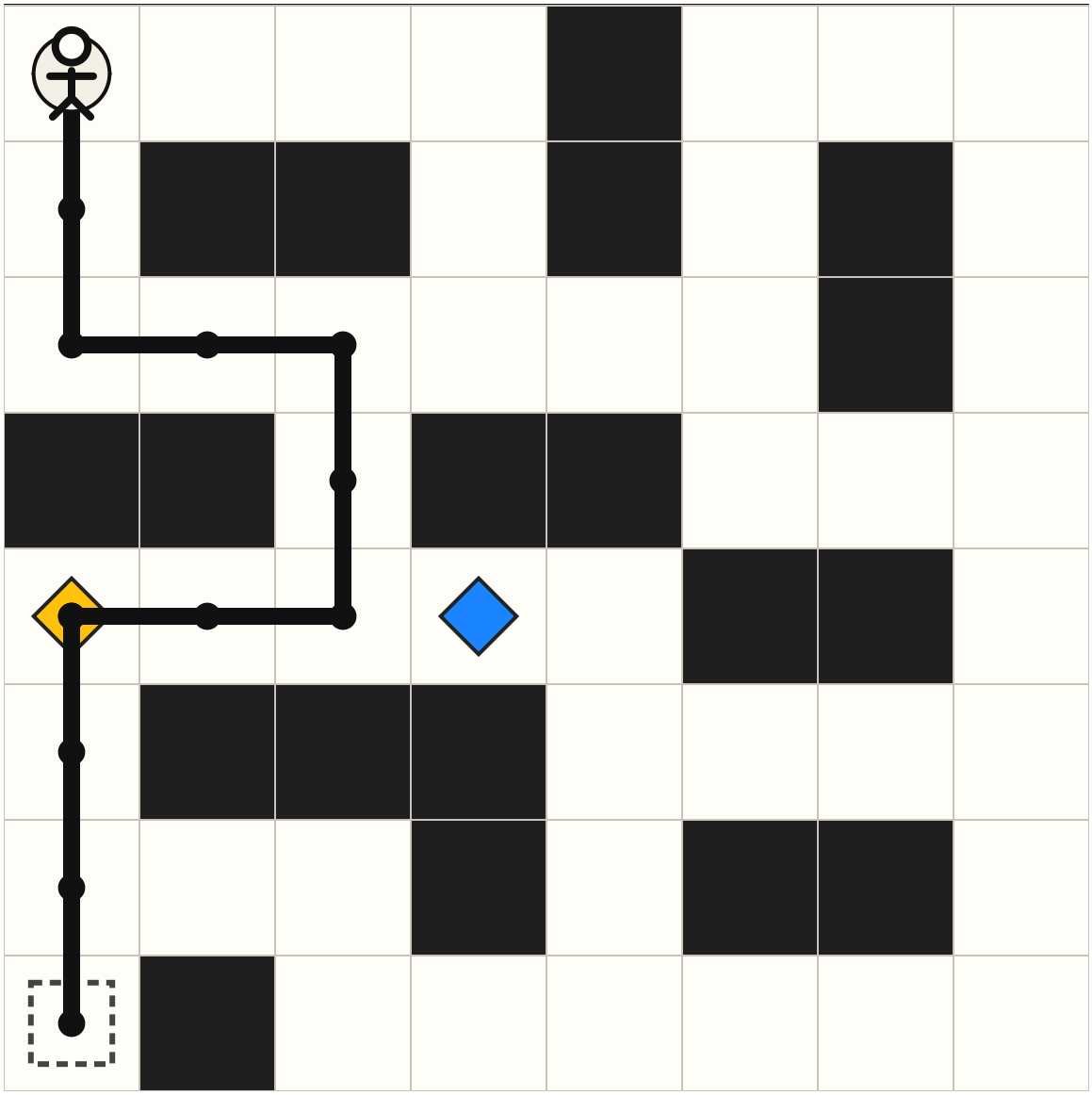
Create the held-out Q2 demonstration task and compute the exact posterior for its sampled trajectory.

In [13]:
```
q2_demo_task = GridworldTask(
    grid=task.grid,
    init_loc=task.init_loc,
    goal_locs=q2_eval_layouts[0],
    beta=task.beta,
)
q2_demo_task.validate()
q2_demo_backend = MemoPolicyBackend(q2_demo_task)
q2_demo_backend.build()

q2_demo = sample_trajectory(q2_demo_task, horizon=HORIZON, rng=np.random.def
q2_demo_exact = online_goal_posteriors(q2_demo_task, q2_demo.action_indices,

q2_frames = np.stack(
```

```
    [encode_grid_frame(q2_demo_task, position=q2_demo.positions[step], step=
     for step in range(len(q2_demo.action_indices))],
    axis=0,
)[None]
q2_prev_actions = np.full((1, len(q2_demo.action_indices)), q2_demo_task.n_a
q2_prev_actions[0, 1:] = np.asarray(q2_demo.action_indices[:-1], dtype=np.ir
q2_action_indices = np.asarray(q2_demo.action_indices, dtype=np.int64)
q2_frames_t = torch.tensor(q2_frames, dtype=torch.float32, device=device)
q2_prev_actions_t = torch.tensor(q2_prev_actions, dtype=torch.long, device=d

for run in q2_example_runs.values():
    run['model'].eval().to(device)

with torch.no_grad():
    q2_action_gru_out = q2_example_runs['action_gru']['model'](q2_frames_t,
    q2_conv_gru_out = q2_example_runs['conv_gru']['model'](q2_frames_t, q2_p

q2_action_gru_online = torch.softmax(q2_action_gru_out['posterior_logits'],
q2_conv_gru_online = torch.softmax(q2_conv_gru_out['posterior_logits'], dim=

q2_policy_action_probs = []
with torch.no_grad():
    for goal_index in range(q2_demo_task.n_goals):
        goal_ids = torch.tensor([goal_index], dtype=torch.long, device=devic
        logits = q2_example_runs['goal_conditioned_policy']['model'](q2_fram
        probs = torch.softmax(logits, dim=-1)[0].detach().cpu().numpy()
        q2_policy_action_probs.append(probs)
q2_policy_action_probs = np.asarray(q2_policy_action_probs)
q2_policy_online = online_posteriors_from_goal_conditioned_action_probs(q2_p

print('held-out random-layout goals :', q2_demo_task.goal_locs)
print('held-out goal index          :', q2_demo.goal_index)
print('held-out actions             :', q2_demo.action_indices)
print('exact final posterior        :', np.round(q2_demo_exact[-1], 3))
display(SVG(render_gridworld_svg(q2_demo_task, trajectory=q2_demo, cell_size
```

```
held-out random-layout goals : ((np.int64(0), np.int64(0)), (np.int64(4), n
p.int64(0)), (np.int64(4), np.int64(3)))
held-out goal index          : 0
held-out actions             : [1, 1, 1, 0, 4, 4, 1, 1, 3, 3, 1, 1]
exact final posterior        : [1. 0. 0.]
```

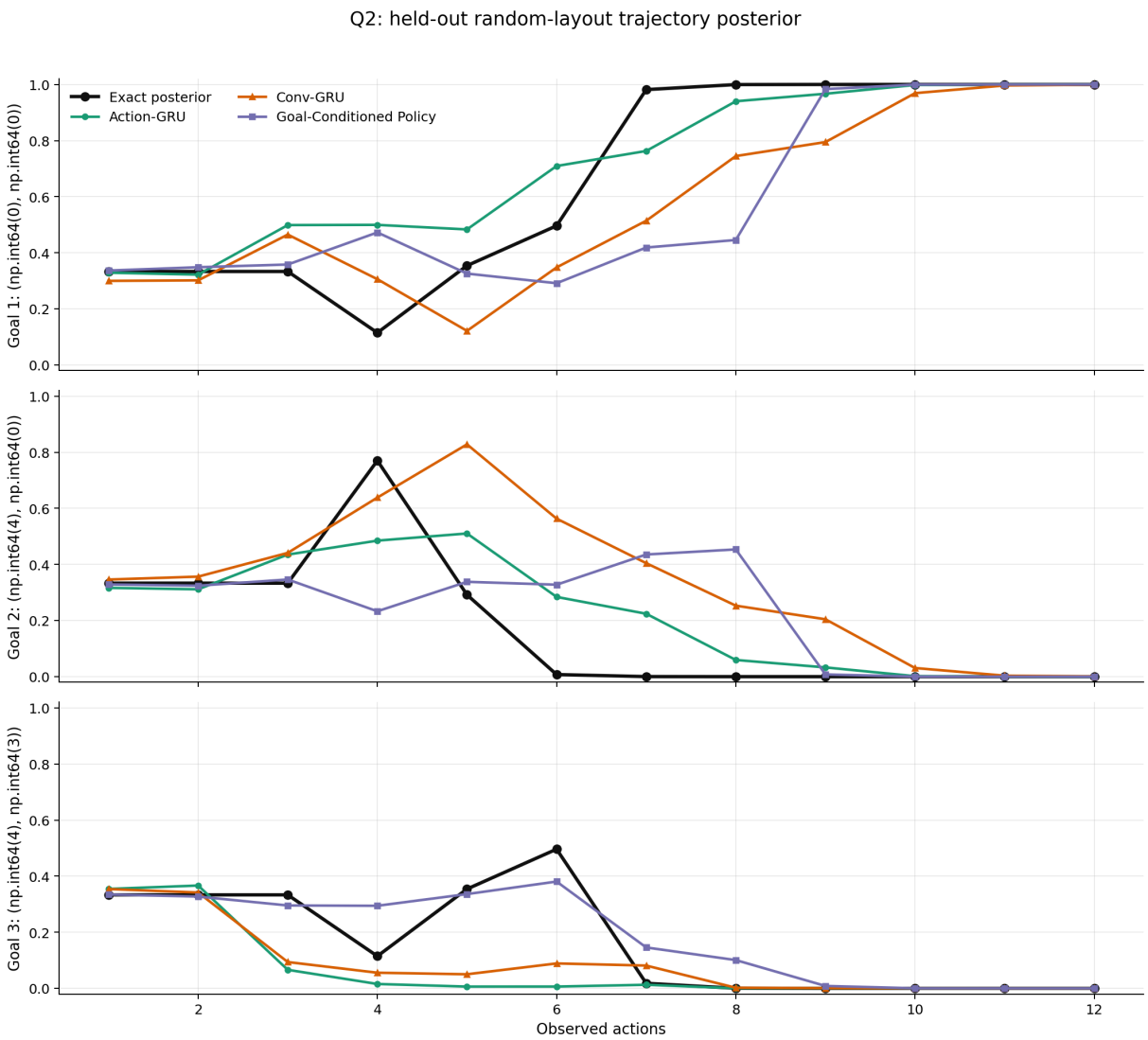
Q2 held-out random-layout trajectory



Plot the Q2 demonstration posterior in the same format as Q1.

In [14]:
```
q2_timesteps = np.arange(1, len(q2_demo.action_indices) + 1)
q2_goal_labels = [f'Goal {idx + 1}: {goal}' for idx, goal in enumerate(q2_de
fig, axes = plt.subplots(q2_demo_task.n_goals, 1, figsize=(12, 3.5 * q2_demo
if q2_demo_task.n_goals == 1:
    axes = [axes]

for goal_index, ax in enumerate(axes):
    ax.plot(q2_timesteps, q2_demo_exact[:, goal_index], marker='o', linewidt
    for variant, series in [
        ('action_gru', q2_action_gru_online[:, goal_index]),
        ('conv_gru', q2_conv_gru_online[:, goal_index]),
        ('goal_conditioned_policy', q2_policy_online[:, goal_index]),
    ]:
        style = MODEL_STYLES[variant]
        ax.plot(q2_timesteps, series, marker=style['marker'], markersize=4,
    ax.set_ylim(-0.02, 1.02)
    ax.set_ylabel(q2_goal_labels[goal_index])
```

```
    ax.grid(alpha=0.25)
    if goal_index == 0:
        ax.legend(loc='upper left', ncol=2)

axes[-1].set_xlabel('Observed actions')
fig.suptitle('Q2: held-out random-layout trajectory posterior', y=1.02, font
fig.tight_layout()
plt.show()
```



6. Question 3: How does rationality change the task?

Q3 keeps the fixed default layout from Q1 and varies only the Boltzmann rationality parameter β.

β	Agent behaviour	Inference effect
0.75	Noisy	Trajectories are less decisive; the target posterior stays diffuse.

β	Agent behaviour	Inference effect
1.5	Moderately goal-directed	Evidence is useful but still imperfect.
3.0	Strongly goal-directed	Actions are highly informative once the policy is learned.

Train and evaluate the models separately for each rationality value, using the same default-layout protocol as Q1.

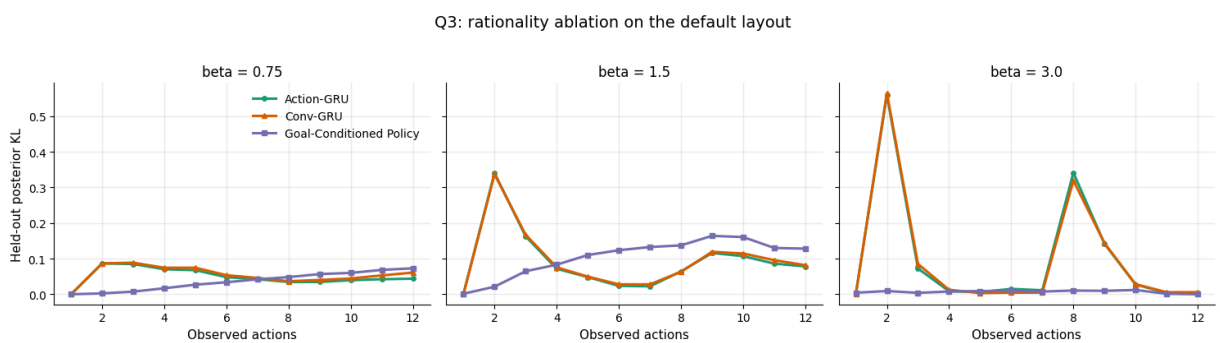
In [15]:
```
beta_values = [0.75, 1.5, 3.0]
beta_train_episodes = 1024
beta_eval_episodes = 512
beta_results = []

for beta in beta_values:
    beta_task = make_default_task(beta=beta)
    beta_backend = MemoPolicyBackend(beta_task)
    beta_backend.build()
    beta_tag = str(beta).replace('.', 'p')

    _, beta_train_npz, _ = collect_and_save(
        f'q3_beta_{beta_tag}_train', beta_task, beta_backend, beta_train_epi
    )
    _, beta_eval_npz, _ = collect_and_save(
        f'q3_beta_{beta_tag}_eval', beta_task, beta_backend, beta_eval_episc
    )

    beta_runs = train_model_suite(
        beta_train_npz,
        eval_npz=beta_eval_npz,
        epochs=DEFAULT_EPOCHS,
        prefix=f'q3_beta_{beta_tag}',
    )
    for variant, run in beta_runs.items():
        metrics = run['metrics']
        beta_results.append({
            'beta': beta,
            'variant': variant,
            'posterior_kl': metrics['posterior_kl'],
            'posterior_kl_by_step': metrics['posterior_kl_by_step'],
            'final_loss': float(run['losses'][-1]),
        })

print(f"{'beta':<8} | {'model':<24} | mean KL")
print('-' * 46)
for row in beta_results:
    print(f"{row['beta']:<8} | {row['variant']:<24} | {row['posterior_kl']:.
```

```
q3_beta_0p75_train            1024 episodes | goal counts: {0: 342, 1: 346,
2: 336}
q3_beta_0p75_eval              512 episodes | goal counts: {0: 162, 1: 157,
2: 193}
q3_beta_1p5_train             1024 episodes | goal counts: {0: 356, 1: 350,
2: 318}
```

```
q3_beta_1p5_eval               512 episodes | goal counts: {0: 161, 1: 177,
2: 174}
q3_beta_3p0_train             1024 episodes | goal counts: {0: 365, 1: 321,
2: 338}
q3_beta_3p0_eval               512 episodes | goal counts: {0: 167, 1: 178,
2: 167}
beta     | model                     | mean KL
----------------------------------------------
0.75     | action_gru                | 0.0496
0.75     | conv_gru                  | 0.0550
0.75     | goal_conditioned_policy   | 0.0364
1.5      | action_gru                | 0.0935
1.5      | conv_gru                  | 0.0967
1.5      | goal_conditioned_policy   | 0.1048
3.0      | action_gru                | 0.0994
3.0      | conv_gru                  | 0.0983
3.0      | goal_conditioned_policy   | 0.0072
```

Plot the posterior-KL curves for the three rationality settings.

In [16]:
```
fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharey=True)
for ax, beta in zip(axes, beta_values):
    rows = [row for row in beta_results if row['beta'] == beta]
    for row in rows:
        style = MODEL_STYLES[row['variant']]
        timesteps = np.arange(1, len(row['posterior_kl_by_step']) + 1)
        ax.plot(timesteps, row['posterior_kl_by_step'], marker=style['marker
    ax.set_title(BETA_STYLES[beta]['label'])
    ax.set_xlabel('Observed actions')
    ax.grid(alpha=0.25)
axes[0].set_ylabel('Held-out posterior KL')
axes[0].legend(loc='upper right')
fig.suptitle('Q3: rationality ablation on the default layout', y=1.03, fonts
fig.tight_layout()
plt.show()
```



Q3 conclusion. Rationality changes the target posterior as well as the trajectories. At $\beta = 0.75$, behaviour is noisy and the exact posterior remains relatively diffuse, so KL can be small even when the trajectory is not very informative. At $\beta = 1.5$, the posterior sharpens but the actions still contain noise, making the task harder for these small models. At $\beta = 3.0$, the goal-conditioned policy model performs best because near-optimal actions are easy to score under the correct goal.

The main takeaway is that higher rationality makes actions more informative, and the generative policy model benefits most because its inference procedure matches the Bayesian observer.

Overall takeaway. The experiments separate three sources of difficulty. Held-out trajectories in a fixed layout are handled well; random goal layouts introduce a real generalization gap; and rationality changes how sharply actions identify the goal. The goal-conditioned policy model is strongest when the layout is fixed and actions are reliable, while the Conv-GRU is the most promising baseline under random-layout shift.